NetStar Ticketsystem – Tagesprojekt (Aufgabenblatt)

Oct 11, 2026 · @Christopher Partridge

Ausgangslage

Die NetStar GmbH betreut die IT mehrerer Kunden und bekommt täglich Störungsmeldungen (Tickets). Heute baut ihr dafür ein kleines Ticketsystem als Konsolenanwendung in Java: Vormittags verwaltet ihr die Tickets mit Collections, nachmittags wertet ihr sie mit Lambdas und Streams aus.

Das bekommt ihr fertig:

- `Ticket.java` : die Klasse für ein Ticket (Id, Titel, Kunde, Bearbeiter, Priorität, Status)
- `Beispieldaten.java` : 10 Beispiel-Tickets
- `Main.java` : ein Gerüst mit dem Menü und `TODO` -Markierungen

Das schreibt ihr selbst:

- `TicketSystem.java` mit den Collections (Teil A)
- `Auswertung.java` mit Lambdas und Streams (Teil B)

Priorität 1 bedeutet kritisch, 4 bedeutet niedrig. Der Status ist einer der Texte `OFFEN` , `IN_BEARBEITUNG` oder `ERLEDIGT` .

Tagesablauf

Die Zeiten sind Richtwerte. Wer schneller ist, macht mit den Zusatzaufgaben weiter, wer länger braucht, lässt Zusatzaufgaben aus.

Block	Inhalt	Richtwert
1	A1 und A2: Tickets speichern, anzeigen, neu anlegen	60 min
2	A3 und A4: Warteschlange (Queue) und Rückgängig (Stack)	60 min
3	A5 und A6: Map pro Bearbeiter, Set für Kunden, Menü fertigstellen	60 min
Pause	Nach Absprache	
4	B1 bis B3: Predicates und erste Streams	60 min
5	B4 bis B5: Sortieren, Filtern nach Kunde, `anyMatch`	60 min
6	B6 und B7: Gruppieren und Durchschnitt, Aufräumen	60 min
7	Abnahme mit der Checkliste, Zusatzaufgaben	Rest

Teil A (Vormittag): Collections

Ihr schreibt die Klasse `TicketSystem` . Sie verwaltet alle Tickets und benutzt dafür fünf verschiedene Collections. Alle Prüfwerte gelten direkt nach dem Programmstart mit den unveränderten Beispieldaten.

Nr.	Das tut ihr	So prüft ihr es

A1	Legt in `TicketSystem` eine `List<Ticket>` an und schreibt `hinzufuegen(Ticket ticket)`. In `Main` fügt ihr alle Beispiel-Tickets hinzu. Menüpunkt 1 zeigt alle Tickets mit einer Schleife an.	10 Zeilen, Ticket #1 steht oben
A2	Schreibt `anlegen(titel, kunde, bearbeiter, prioritaet)`. Die neue Id ist die größte bisherige Id plus 1, der Status ist `OFFEN`. Menüpunkt 2 fragt die vier Werte ab.	Das neue Ticket hat die Id 11 und erscheint in Menüpunkt 1
A3	Offene Tickets warten in einer `Queue` (`LinkedList`). Beim Hinzufügen kommen nur Tickets mit Status `OFFEN` hinein. Schreibt `naechstesErledigen()` mit `poll()`: Das Ticket bekommt den Status `ERLEDIGT`. Ist die Queue leer, gebt eine Meldung aus.	Erstes Erledigen: #1, zweites Erledigen: #2
A4	Erledigte Tickets kommen zusätzlich auf einen `Stack`. `letzteErledigungRueckgaengig()` holt das oberste Ticket mit `pop()`, setzt es auf `OFFEN` und stellt es hinten in die Queue.	Zweimal erledigen, einmal rückgängig: #2 ist wieder `OFFEN`, #1 bleibt `ERLEDIGT`
A5	Legt eine `Map<String, List<Ticket>>` an (Bearbeiter, seine Tickets) und ein `Set<String>` für die Kunden. Menüpunkt 5 zeigt die Tickets pro Bearbeiter, Menüpunkt 6 alle Kunden.	Anna Berger 3, Lena Fischer 3, Tom Weber 4 Tickets; 4 Kunden, jeder nur einmal
A6	Macht das Menü fertig. Eine falsche Eingabe (Text statt Zahl, leere Queue) darf das Programm nicht abstürzen lassen.	Alle Menüpunkte 1 bis 6 laufen mehrfach hintereinander

Tipps:

- Welche Collection wofür? Liste = alle Tickets in Reihenfolge, Queue = wer zuerst kommt, wird zuerst bearbeitet, Stack = zuletzt erledigt, zuerst rückgängig, Map = Schlüssel und Wert, Set = jeder Eintrag nur einmal.
- Hat ein Bearbeiter in der Map noch keine Liste, legt zuerst eine neue `ArrayList` an und speichert sie in der Map (`containsKey` , `put`).
- `poll()` gibt bei einer leeren Queue `null` zurück. Prüft das, bevor ihr mit dem Ticket arbeitet.

Teil B (Nachmittag): Lambdas und Streams

Ihr schreibt die Klasse `Auswertung` . Sie bekommt im Konstruktor die Liste aller Tickets (`system.getTickets()`). Alle Regeln schreibt ihr als ausgeschriebene Lambda-Ausdrücke, Methodenreferenzen mit `::` sind nicht erlaubt. Die Prüfwerte gelten wieder direkt nach dem Programmstart.

Nr.	Das tut ihr	So prüft ihr es
B1	Legt als Attribute benannte Lambdas an: `Predicate<Ticket> istOffen` (Status ist `OFFEN`), `Predicate<Ticket> istKritisch` (Priorität 1), `Function<Ticket, String> titelMitBearbeiter` (Titel plus Bearbeiter in Klammern), `Consumer<String> zeileAusgeben` (Zeile mit `-` davor).	Das Programm kompiliert, die Variablen kommen in B2 bis B4 zum Einsatz
B2	`long anzahlOffen()` mit `filter` und `count`.	6
B3	`List<String> kritischeOffeneTitel()` mit zwei `filter`, `map` und `collect`. Außerdem `offeneTicketsAusgeben()`: alle offenen	Mailserver nicht erreichbar, Backup schlaegt fehl; 6 Ausgabezeilen

	Tickets mit `titelMitBearbeiter` umwandeln und mit `forEach(zeileAusgeben)` ausgeben.	
B4	`List<Ticket> dringendsteOffene(int anzahl)`: offene Tickets mit `sorted` nach Priorität sortieren, bei Gleichstand nach Id, dann `limit(anzahl)`.	Für 3: #2, #5, #1
B5	`List<String> titelVonKunde(String kunde)`, alphabetisch sortiert. Dazu `boolean hatKritischesOffenesTicket(String kunde)` mit `anyMatch`.	Autohaus Kraemer: Monitor flackert, Server-Festplatte fast voll, WLAN im Besprechungsraum langsam; kritisch offen: Schmidt & Soehne `true`, Autohaus Kraemer `false`
B6	`Map<String, Long> anzahlProBearbeiter()` mit `Collectors.groupingBy` und `Collectors.counting()`.	Anna Berger 3, Lena Fischer 3, Tom Weber 4
B7	`double durchschnittPrioritaet()` mit `mapToInt` und `average`. Baut danach Menüpunkt 7 so, dass er alle Auswertungen ausgibt, und räumt euren Code auf (sprechende Namen, keine doppelten Lambdas).	2,3

Tipps:

- Ein Stream macht nichts, solange eine terminale Operation fehlt (`count` , `collect` , `forEach` , `anyMatch`).
- Ein Stream kann nur einmal benutzt werden. Für jede Auswertung erzeugt ihr einen neuen mit `tickets.stream()` .
- In B4 kann der Compiler den Typ nicht selbst erkennen. Schreibt den Typ beim ersten Vergleich dazu: `Comparator.comparing((Ticket t) -> t.getPrioritaet())` , danach geht `.thenComparing(...)` .
- Ein benanntes Lambda benutzt ihr direkt: `tickets.stream().filter(istOffen)` . Wollt ihr es in einem Lambda aufrufen, nehmt `istOffen.test(ticket)` .

Zusatzaufgaben

Nur für alle, die A und B fertig haben und die Checkliste erfüllen. Sie sind freiwillig.

- **Z1: Dringende Tickets zuerst.** Die Queue arbeitet nach „wer zuerst kommt“. Ersetzt sie durch eine `PriorityQueue<Ticket>` , die kritische Tickets zuerst herausgibt. Den `Comparator` schreibt ihr mit einem Lambda.
- **Z2: Offene Tickets pro Kunde.** Gebt aus, wie viele offene Tickets jeder Kunde hat (`filter` , `groupingBy` , `counting`).
- **Z3: Tickets suchen.** Neuer Menüpunkt: Der Benutzer gibt einen Suchtext ein, ausgegeben werden alle Tickets, deren Titel den Text enthalten (Groß- und Kleinschreibung egal).
- **Z4: Erstes kritisches Ticket eines Bearbeiters.** Mit `filter` und `findFirst` bekommt ihr ein `Optional<Ticket>` . Gebt das Ticket aus oder „Keine kritischen Tickets“, wenn keines existiert.

Abgabe und Regeln

Am Ende zeigt ihr der Lehrkraft euer laufendes Programm und geht die Abnahme-Checkliste durch. Wie ihr den Quellcode abgebt, legt die Lehrkraft fest.

- Lambdas ausschreiben, keine Methodenreferenzen mit `::` .

- Keine Umlaute und kein ß in Texten und Namen im Quelltext (ae, oe, ue, ss), sonst zeigt die Windows-Konsole Zeichensalat. Die Beispieldaten sind schon so geschrieben.
- `Ticket.java` und `Beispieldaten.java` bleiben unverändert.
- Ein Programm, das nicht kompiliert, ist nichts wert. Kompiliert deshalb nach jeder Teilaufgabe und testet sofort.
- Bei Problemen: erst Tipps und Mitschriften, dann die Mitschüler, dann die Lehrkraft.

© Christopher Partridge